

Stream

Stream

Come trasformare e filtrare collezioni con una sintassi dichiarativa.

Cosa sono gli stream

Uno stream è un modo per lavorare su una sequenza di dati dichiarando cosa vuoi ottenere.

Invece di scrivere ogni passaggio con un ciclo, componi operazioni come:

- filtra
- trasforma
- raccogli il risultato

Gli stream non sostituiscono sempre i cicli. Sono uno strumento in più, utile quando il flusso di trasformazione è chiaro.

Da lista a stream

```
public class EsempioStream {  
    public static void main(String[] args) {  
        ArrayList nomi = new ArrayList<>();  
        nomi.add("Luca");  
        nomi.add("Sara");  
        nomi.add("Mina");  
  
        List risultato = nomi.stream()  
            .filter(nome -> nome.length() > 4)  
            .toList();  
  
        System.out.println(risultato);  
    }  
}
```

Output:

```
[Luca, Sara]
```

`Mina` ha 4 caratteri, quindi viene esclusa dalla condizione `> 4`.

Collezione e stream

Una collezione, come `ArrayList`, contiene davvero i dati.

Uno stream è un flusso di lavoro su quei dati.

```
nomi.stream()
```

crea uno stream a partire dalla lista.

Alla fine, con `toList()`, raccogli il risultato in una nuova lista.

filter

`filter` tiene solo gli elementi che rispettano una condizione.

```
List numeri = List.of(3, 8, 1, 10, 5);

List grandi = numeri.stream()
    .filter(numero -> numero >= 5)
    .toList();

System.out.println(grandi);
```

Output:

```
[8, 10, 5]
```

La lambda deve restituire `true` o `false` .

map

`map` trasforma ogni elemento.

```
List nomi = List.of("luca", "sara", "mina");

List maiuscoli = nomi.stream()
    .map(nome -> nome.toUpperCase())
    .toList();

System.out.println(maiuscoli);
```

Output:

```
[LUCA, SARA, MINA]
```

Il numero di elementi resta lo stesso, ma ogni elemento viene trasformato.

Combinare filter e map

```
List nomi = List.of("luca", "sara", "anna", "mario");

List risultato = nomi.stream()
    .filter(nome -> nome.length() == 4)
    .map(nome -> nome.toUpperCase())
    .toList();

System.out.println(risultato);
```

Output:

```
[LUCA, SARA, ANNA]
```

Prima filtriamo i nomi lunghi 4 caratteri. Poi li trasformiamo in maiuscolo.

Quando usare uno stream

Uno stream è adatto quando stai descrivendo una trasformazione sui dati.

Un ciclo normale può essere migliore quando:

- la logica ha molti passaggi
- devi modificare variabili esterne
- il codice con stream diventa troppo difficile da leggere

Regola pratica

Se puoi leggere la catena ad alta voce, lo stream è probabilmente chiaro:

"prendi i nomi, tieni quelli lunghi 4, trasformali in maiuscolo, raccoglili in una lista".

Se devi fermarti a decifrare ogni pezzo, usa un ciclo più esplicito.